



Sonic Sentinel AI

THEME: AcousticX Intelligence

GROUP MEMBERS:
Sir Hamza Ali (Mentor)
Moiz Jan (leader)
Sarim Owais
Ibad
Muhammad Hamza
Zarwasham Lodhi
Alishba

CATEGORY: NEXT WAVE AI and ML
VERSION: 1.0

Table of content:

1.1 Background and Necessity for the Application

1.2

Proposed Solution

- Reference Application
- Development Phase of the Application

1.3 Purpose of the Document

1.4 Scope of Project

1.5 Constraints

1.6 Requirements

1.7 Non-Functional Requirements

1.8 Competition Integrity and Anti-Shortcut Requirements

1.9 Interface Requirements

- 1.9.1 Hardware
- 1.9.2 Software

1.10 Project Deliverables

- i. Project Report
- ii. Source Code
- iii. Dataset
- iv. Python Classification Model Evidence
- v. Google Teachable Machine Evidence
- vi. Model Prediction and Confidence Comparison Report
- vii. Alert Rule Files
- viii. Test Cases
- ix. Installation Instructions
- x. Execution Instructions
- xi. GitHub Repository
- xii. Deployed Application
- xiii. Demonstration Video
- xiv. Technical Blog
- xv. AI Tool Usage Declaration
- xvi. Final Submission Checklist

1.1 Background and Necessity for the Application:

Monitoring sounds in places like manufacturing plants, public areas, and secure buildings is extremely important. When critical sounds happen such as a machine breaking down, glass shattering, a gunshot, or someone screaming for help it usually means there is an emergency or threat that requires immediate action.

The traditional way of handling this relies on people sitting and listening to live audio or manually checking recordings after an event has already occurred. This manual process takes way too much time, and it is very easy for a human to miss a sound, react too late, or trigger a false alarm, especially if the environment is noisy or crowded.

That is exactly why we are building 'SonicSentinel AI'. We want to automate and improve this process using Python web development, machine learning, and Google Teachable Machine. Instead of relying entirely on human monitoring, our application automatically analyzes uploaded audio clips or live microphone feeds.

The system is designed to detect ten specific sounds: Machinery Faults, Glass Breaking, Alarms or Sirens, Vehicle Horns, Animal Sounds, Gunshots, Panic Screams, Aggression, a Person Asking for Help, and Background Noise.

Whenever it detects a sound, a Python AI model generates a confidence score, and we compare that result with a separate Google Teachable Machine model to double-check the accuracy. The application also checks the audio quality, handles overlapping noises, assigns severity levels, and immediately sends out alerts for critical events. Ultimately, the goal is to provide a fast and accurate tool to help people make better decisions, though it is strictly a competition prototype and not an official, certified emergency-response system



1.2 Proposed Solution:

For our project, we are developing an AI-powered, web-based application that detects sound events in real-time to help users, security teams, and maintenance staff identify important acoustic triggers quickly and accurately.

We are designing the system to accept two types of inputs: users can either upload pre-recorded audio clips or turn on live microphone monitoring directly through the web interface. Once the audio is captured, our backend pipeline cleans it up by validating the file, removing unwanted background noise, dividing the audio into fixed-duration segments, and extracting the necessary acoustic features for analysis.

To make sure our predictions are reliable, we are using a dual-model approach. First, our custom Python classification model analyzes the processed audio and predicts the sound category while generating a confidence score. At the same time, we feed the exact same audio segment into a separately trained Google Teachable Machine (GTM) model, which independently generates its own prediction and confidence scores. Our application then compares the results from both models to confirm the event.

Alongside the predictions, our app will generate visual waveforms and spectrograms, check the overall recording quality, handle overlapping sounds, and apply specific event rules to trigger alerts for critical situations.

Our system is required to accurately detect the following 10 mandatory sound categories:

- Machinery Fault
- Glass Breaking
- Alarm or Siren
- Vehicle Horn
- Animal Sound
- Gunshot
- Panic Scream
- Aggression or Violent Conflict
- Person Asking for Help
- Background Noise

We also have the option to add extra categories later, like drone sounds, explosions, or fireworks, depending on how our dataset shapes up. Finally, it is important to note in our documentation that we are building this as a competition prototype for ethical and controlled testing, so it is not meant to replace certified emergency-response or law-enforcement systems. The flow chart for the sonic Sential AI is given below:

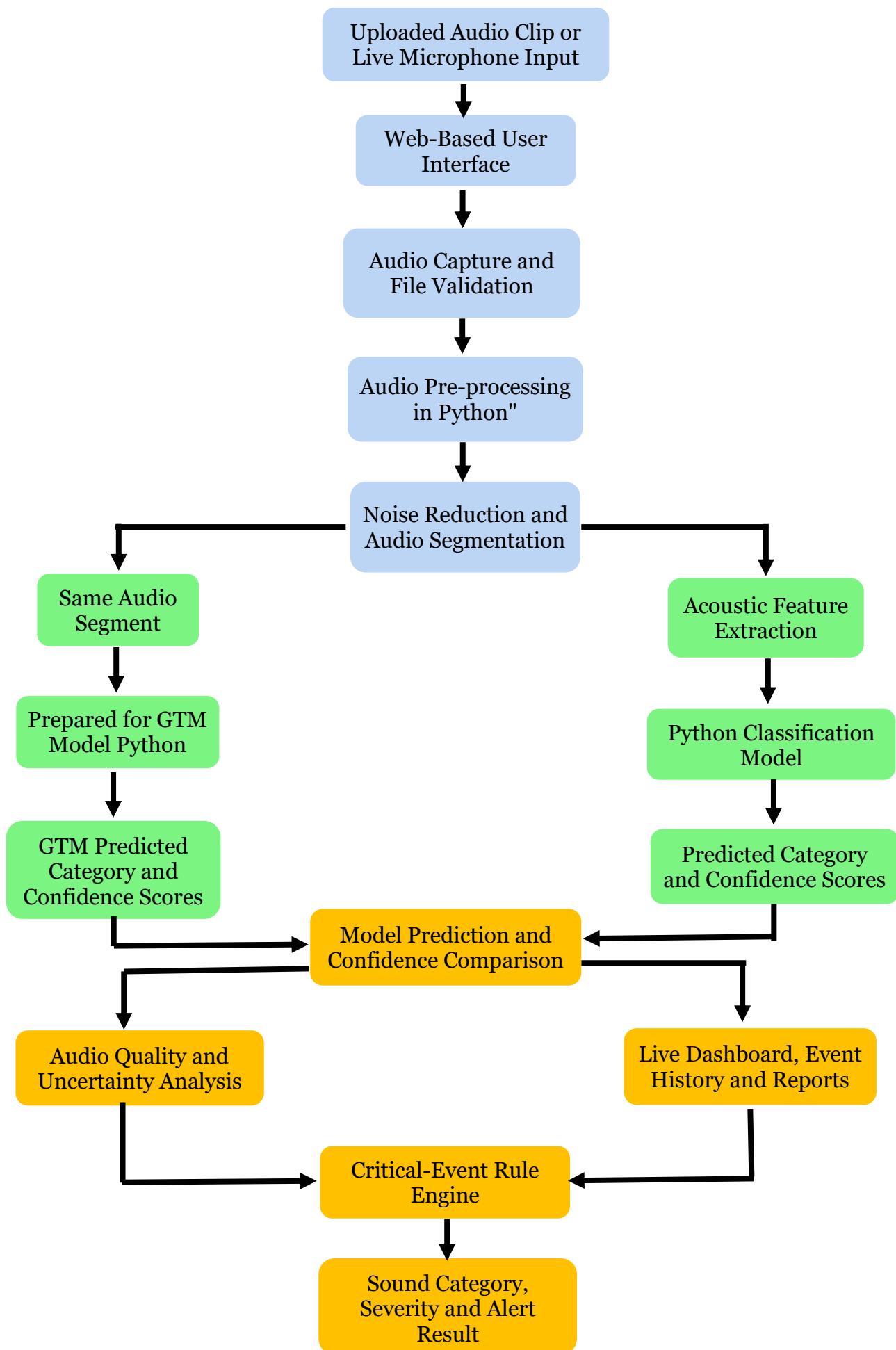


FIGURE 01: Sample Architecture of Application

1.3 Purpose Of the Document:

The main purpose of this document is to provide a clear, step-by-step development plan for our Sonic Sentinel AI application. It serves as a complete roadmap for our team to build a reliable sound-event detection system using Artificial Intelligence, Machine Learning, Python audio processing, and Google Teachable Machine. As we bring this to life using a Flask backend and a Streamlit web interface, this document ensures that everyone involved from our development team to the project stakeholders has a shared understanding of the exact requirements and goals.

This document highlights the core objectives of our project, specifically how we plan to identify critical sounds in real time and reduce the need for humans to manually listen to audio feeds. It details how our system will generate confidence-based predictions and compare the results between our custom Python model and the Google Teachable Machine model to ensure accuracy. Finally, it outlines how the application will trigger fast alerts for emergencies like gunshots, glass breaking, panic screams, aggression, machinery faults, and people asking for help. Ultimately, this is a guiding document meant to keep both the developers building the application and the stakeholders reviewing it on the exact same page.

1.4 Scope of the Document:

For the scope of our project, we are building a complete Machine Learning pipeline for the Sonic Sentinel AI application. The main goal is to fully automate sound-event detection so that the system can instantly analyze uploaded audio clips or live microphone feeds using audio-processing and machine learning techniques.

Within this scope, our application will handle the background work of extracting acoustic features, creating visual waveforms and spectrograms, and classifying the audio using both a Python model and Google Teachable Machine. Our system will compare the confidence scores of these two models to accurately identify ten specific sounds: machinery faults, glass breaking, alarms, vehicle horns, animal sounds, gunshots, panic screams, aggression, calls for help, and background noise.

Finally, the scope includes building a complete and user-friendly web interface that triggers alerts based on event severity, routes any uncertain or confusing sounds for manual human review, and displays live detections, event history, and detailed reports for the users.

1.5 Constraints:

Our project has a few practical limitations and challenges that we need to keep in mind. First off, the app might struggle if the audio quality is bad, if there is too much background noise, if the sound source is too far away, or if multiple sounds are overlapping at the exact same time. Even the specific type of microphone a user has can change how well a sound is picked up.

How accurate our application is will depend heavily on our training data if our dataset isn't big, balanced, and diverse enough, or if we are short on computing power, the models just won't perform well. Another challenge is that our custom Python model and the Google Teachable Machine model might end up giving us completely different predictions or confidence scores for the exact same audio clip.

Finally, we have to take privacy and fairness seriously. We must get proper user consent before accessing any microphone to protect audio privacy, and we have to make sure our system isn't biased toward specific environments, recording devices, or people's voices.

1.6 Functional Requirement:

For this section, the original document lists 80 specific technical requirements. To make it easier to read for our project report, I have grouped these requirements into the main features our Sonic Sentinel AI application actually needs to perform:

User Accounts and Access:

- The application must have a secure registration and login system.
- It needs role-based access so regular users, reviewers, security staff, and administrators all have the correct permissions.
- Every user will get a unique ID and a profile they can manage.

Audio Input and Handling:

- Users must be able to upload single or multiple audio files in formats like WAV, MP3, or FLAC.
- The app must support live microphone monitoring and clearly show if the mic is active, paused, or disconnected.
- Before doing anything, the system must validate the audio to ensure it isn't corrupted, empty, or the wrong format.
- Users should be able to play, pause, and preview the audio directly in the app, and the system needs to extract basic file details like duration and size.

Cleaning and Processing the Audio:

- The backend will clean up the audio using Python by normalizing the volume, trimming silence, and reducing background noise.
- Long recordings or live mic feeds will be chopped into smaller, fixed-duration segments for the AI to analyze.
- The app must be smart enough to detect severe audio clipping or too much background noise and flag the audio quality as Good, Acceptable, Poor, or Unusable.

AI Models and Predictions:

- The system will extract acoustic features (like MFCCs and Chroma) and generate visual Waveforms and Spectrograms for the users to see.
- We will train and compare at least three Python classification models, and the best one will be used to predict the sound and generate a confidence score.
- At the same time, a separate Google Teachable Machine (GTM) model will analyze the exact same audio segment and give its own independent prediction and confidence score.

Comparing Results and Handling Uncertainty:

- The app will compare the predictions and confidence scores of both models.
- If the models disagree, if the confidence is too low, if sounds are overlapping, or if the audio quality is terrible, the app will mark the result as "Uncertain".
- Critical events will need to be confirmed (like detecting a sound twice in a row) before jumping to conclusions to avoid false alarms.

Alerts and Event Rules:

- The system will assign severity levels to sounds. For example, background noise is just informational, but glass breaking, gunshots, or a person asking for help will trigger a high or critical alert.
- Administrators can configure these rules, such as changing the minimum confidence needed to trigger an alert.
- Authorized users can acknowledge, dismiss, or escalate these alerts on the dashboard.

Manual Review System:

- Any confusing or uncertain sounds are sent to a manual review queue.
- An authorized human reviewer can listen to the audio, confirm or correct what the AI guessed, leave comments, and override the AI's decision if it made a mistake.

Dashboards and Reporting:

- We need different dashboards for users, live monitoring, and admins to track recent detections, active alerts, and overall statistics.

- The app will feature an event timeline and allow users to search and filter past events by date, category, or severity.
- Users can download detailed analysis reports (including the spectrograms and model comparisons) or export the data into CSV/Excel files.

System Security and Storage:

- All audio files and database records must be stored securely.
- The system will keep an audit trail of everything tracking logins, uploads, alerts, and model updates.
- It will detect duplicate audio uploads to save space.
- Most importantly for privacy, the app will never secretly record audio and will always inform the user when the microphone is on.

1.7 Non-Functional Requirement:

The non-functional requirements are given below:

1. **Performance (Speed):** The app must process an uploaded 30-second audio clip in 8 seconds or less. For live microphone monitoring, it needs to be even faster and generate a prediction within just 3 seconds so we can react in real time.
2. **Scalability:** The system has to be able to securely store at least 20,000 sound-event records. It also needs to handle multiple users working on the app at the exact same time without lagging or slowing down.
3. **Usability:** The web interface must be intuitive and easy to navigate for everyone, whether it is a regular user, an audio reviewer, a security operator, or a system admin.
4. **Accuracy:** Both the Python and GTM models must hit an overall test accuracy of at least 85% and a macro F1-score of 0.80 or higher. For the most critical emergencies like gunshots, glass breaking, panic screams, aggression, and calls for help the models must specifically maintain a recall of at least 85% so nothing important gets missed.
5. **Availability (Uptime):** The application needs to be highly reliable, staying online and operational with at least a 99% uptime during our required monitoring and evaluation hours.

1.8 Competition Integrity and Anti-Shortcut Requirements:

To prove we actually built this project ourselves and didn't take any shortcuts, we have to follow several strict competition rules.

- **Prove Our Work:** Every team member must be able to fully explain the specific modules they were assigned. The evaluators can pick any function, feature, preprocessing step, or model component and ask us to explain exactly how it works.
- **Consistent Tracking:** We need to push meaningful commits to our GitHub repository across all five days of the competition. We also have to maintain a detailed development log that tracks our completed work, code changes, dataset updates, model failures, tests run, and any problems we face.
- **No Faking It:** We absolutely cannot hard-code test answers, fake our confidence scores, leave code unexplained, or use hidden decision APIs. Doing any of this can get our team disqualified.
- **Surprise Challenges:** We have to be ready for on-the-spot challenges, like making a surprise modification (e.g., adding a new sound category or changing a threshold) or fixing a defect that the judges deliberately put into our application.
- **Blind Testing:** They will test our application using completely unseen audio recordings. These hidden tests will be tricky and might include background noise, echoes, overlapping sounds, low volume, partial events, or distant noises.
- **Strict AI Rules:** If we use any AI tools to help us code during development, we have to clearly declare them in an `AI_USAGE.md` file. AI-generated output cannot replace our own technical understanding, and we are strictly forbidden from using an external generative AI API to make the final sound classifications.
- **Model Independence:** We must prove that our Python model and the Google Teachable Machine model are working completely independently. We cannot feed the Python model's predictions or confidence scores as an input into the GTM model to cheat the results

1.9 Interface Requirement:

To get this system running properly and build it out efficiently, we have some specific hardware and software requirements that our team needs to follow.

1.9.1 Hardware We don't need a supercomputer, but the development and testing machines should at least have an Intel Core i5 or i7 processor and somewhere between 8 to 16 GB of RAM. We also need about 500 GB of hard disk space for storing our datasets, a standard color SVGA monitor, and a basic mouse and keyboard setup.

1.9.2 Software The competition rules give us a list of approved technologies to choose from. For our specific project stack, here is how we are putting it together:

- **Programming & IDE:** Everything will be written in Python, and we are going to use Visual Studio Code as our main development environment.
- **Backend:** We are setting up our server routes and API logic using Flask.
- **Frontend:** We will be building out the user interface and dashboards using Streamlit, supported by standard HTML5, CSS3, and JavaScript where needed.
- **Database:** We will use MongoDB to store all our user data, audio metadata, and event records.
- **Machine Learning:** For the AI side, we are relying on Scikit-learn for our custom Python model, running right alongside the Google Teachable Machine model.
- **Audio Processing:** To extract all those acoustic features from the audio files, we will be heavily using the Librosa library.
- **General Libraries:** We will also need NumPy and Pandas for data handling, and Matplotlib for plotting our waveforms and spectrograms.
- **Testing & Version Control:** We will track all our changes using Git and GitHub, and use tools like py test or Postman to test everything.
- **Deployment:** Once it is ready, we will host the live application on a platform like Streamlit Community Cloud or Render.



1.10 Project Deliverables:

To wrap up our project, Section 1.10 gives us a massive checklist of exactly what we need to submit at the end of the competition. I have broken down the 16 deliverables into a few main categories to make it easier for our team to track:

Documentation and Code

- **Project Report:** We have to submit a complete report that covers everything from the initial problem definition to our database design, architecture, and future enhancements.
- **Source Code & GitHub:** We need to provide a public GitHub repository containing all our clean source code (frontend, backend, AI models, and processing scripts). It is super important that every team member has meaningful commits on the repo.
- **Guides:** We must write clear Installation and Execution instructions so the evaluators know exactly how to set up our environment and use the app.

Data and AI Models

- **The Dataset:** We have to submit our entire organized dataset, including the training, validation, and testing audio files, along with all the metadata.
- **Python Model Evidence:** We need to show proof that we trained and compared at least three different models. This includes our accuracy scores, precision metrics, and the saved model files.
- **GTM Evidence:** We have to provide screenshots of our classes, training configurations, and the exported model from Google Teachable Machine.



Testing and Rules

- **Comparison Report:** We have to write a detailed report testing at least 100 unseen audio recordings to compare how the Python model and the GTM model perform against each other.
- **Alert Rules & Test Cases:** We need to submit our configuration files that trigger alerts, plus proof that we ran extensive test cases (like testing broken audio, overlapping sounds, and fake files).

Media and Final Deliverables

- **Live App:** We need to submit a deployed, public URL of our working application (or provide exact instructions to run it locally if deployment isn't possible).
- **Demonstration Video:** A mandatory .mp4 video showing a complete walkthrough of our app, proving that every single feature works in real-time.
- **Technical Blog:** A detailed 2,000-word blog post discussing the problem we solved, our system architecture, the challenges we faced, and the lessons we learned.
- **AI Usage File:** If we used any AI tools to help us code, we must honestly declare them in an AI_USAGE.md file.
- **Final Checklist:** A final verification that every single item on this list is bundled up and submitted correctly.